

CEN4025C Module 6 Project

For this project we will combine the Angular front end and the Spring Boot back end into a single Maven project to produce one executable JAR containing both applications.

Clone the starter repo and follow the steps below:

Starting project structure

The project is formatted as:

TaskManagerFrontEnd

 cypress

 src

TaskManager

 src

To match the textbook approach, the Angular project must be moved **inside** the Spring Boot project.

Required final structure

Move the TaskManagerFrontEnd project into the TaskManager project (the backend) and rename it to “frontend”.

The structure should become:

TaskManager

 frontend

 cypress

 src

 package.json

 angular.json

 tsconfig.json

 ...

 src

 pom.xml

Update the Angular build configuration

Open the Angular project's package.json file inside TaskManager/frontend.

Change the build script to:

"build": "ng build --configuration production"

This ensures that when Maven runs the Angular build, it creates a production build.

Update Angular production environment settings

Inside the Angular project, create a new file named "environment.prod.ts"

Set it up so production points to the Spring Boot server on port 8080. It should look like this:

```
export const environment = {  
  production: true,  
  apiURL: "http://localhost:8080",  
  authURL: "http://localhost:8080"  
};
```

Add frontend-maven-plugin to pom.xml

Open the pom.xml file in the TaskManager Spring Boot project.

Add the frontend-maven-plugin inside the <build><plugins> section.

Its purpose is to:

- install Node.js and npm locally through Maven
- run npm install
- run npm run build in the Angular app

Use the plugin with the working directory set to frontend.

```
<plugin>  
  <groupId>com.github.eirslett</groupId>  
  <artifactId>frontend-maven-plugin</artifactId>  
  <version>1.12.1</version>  
  <configuration>  
    <workingDirectory>frontend</workingDirectory>  
    <installDirectory>target</installDirectory>  
  </configuration>  
  <executions>  
    <execution>  
      <id>install node and npm</id>  
      <goals>  
        <goal>install-node-and-npm</goal>  
      </goals>  
      <configuration>  
        <nodeVersion>v16.17.0</nodeVersion>  
        <npmVersion>8.19.1</npmVersion>  
      </configuration>  
    </execution>  
    <execution>  
      <id>npm install</id>  
      <goals>  
        <goal>npm</goal>  
      </goals>  
      <configuration>
```

```

        <arguments>install</arguments>
      </configuration>
    </execution>
  </executions>
</plugin>

```

This tells Maven to build the Angular app from the frontend folder during packaging.

Add maven-antrun-plugin to copy Angular build files into Spring Boot

Still in pom.xml, add maven-antrun-plugin below the frontend plugin.

Its job is to copy the Angular production build into Spring Boot's static resource area inside the JAR.

```

<plugin>
  <artifactId>maven-antrun-plugin</artifactId>
  <executions>
    <execution>
      <phase>generate-resources</phase>
      <configuration>
        <target>
          <copy todir="${project.build.directory}/classes/public"
            overwrite="true">
            <fileset dir="${project.basedir}/frontend/dist/task-
manager-front-end/browser"/>
          </copy>
        </target>
      </configuration>
      <goals>
        <goal>run</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

Add Spring MVC configuration to serve Angular

Spring Boot must be configured to host the Angular app.

Create a configuration class in your Spring Boot project, under the config package.

Name the file:

MvcConfig.java

It must implement WebMvcConfigurer and map all unmatched routes to the Angular app in classpath:/public/.

```
package edu.fscj.cen3024c.taskmanager.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import
org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class MvcConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/**")
                .addResourceLocations("classpath:/public/");
    }
}
```

NOTES:

- "/"**" applies to all routes
- "classpath:/public/" points Spring Boot to the copied Angular build files
- requests not handled by Spring Boot should be routed to the Angular application

Verify the modified folder structure

Take a screen snip showing that your project now looks like this conceptually:

TaskManager

```
frontend
  cypress
  src
  package.json
  angular.json
  ...
src
```

pom.xml

That screenshot satisfies the project requirement to show the modified structure.

Run Maven commands

From IntelliJ, use the Maven tool to run:

clean

package

What these commands do

clean removes the previous target folder.

package does the full packaging process:

- installs Node/npm locally through the Maven plugin
- runs npm install
- runs Angular production build
- copies Angular build files into Spring Boot resources
- packages the combined application into one executable JAR

Take screen snips of the results of the package command output (tail end).

Run a test of the packaged application

Run the project in IntelliJ, then access the front end via <http://localhost:8080>

If everything is configured correctly:

- Spring Boot starts on port 8080
- Angular loads from the same server
- the front end communicates with the back end

Take a screen snip of the result.

Submission:

- modified project via GitHub
- (screen snips also via GitHub)
 - folder structure screen snip
 - package screen snip
 - front end execution screensnip